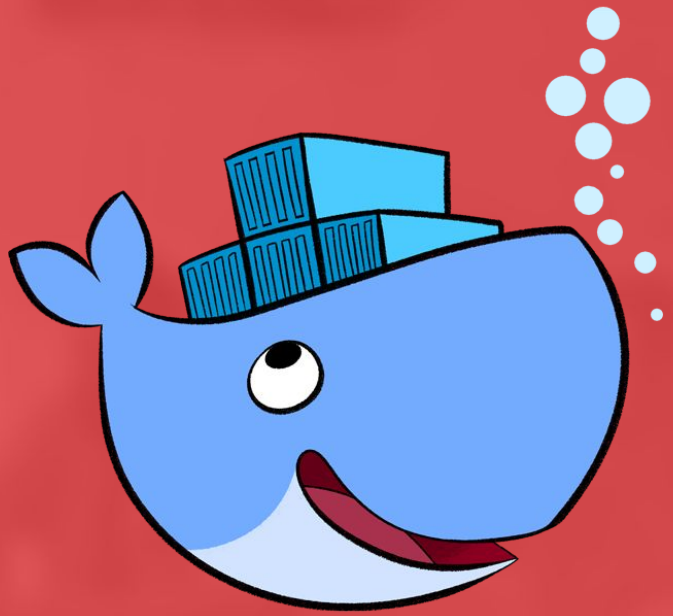
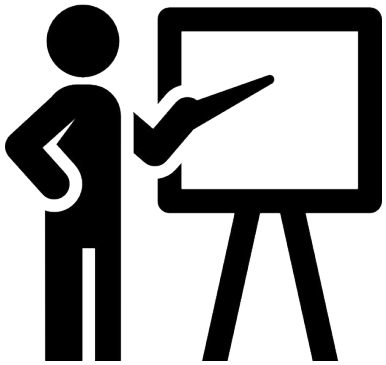


Bienvenue au cours de Docker

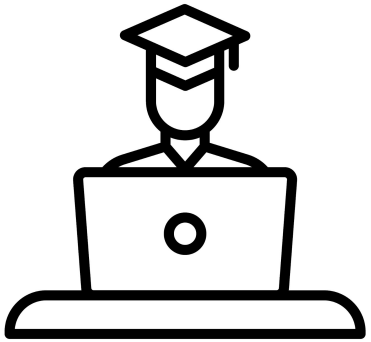
Présentation



[sf≡ir]

Djamel RAAB

Data Engineer



Et vous ?
Prénom
Nom
Job

[≡] School

Déroulement de la formation

Feuille de présence (obligatoire)

C’est quand la pause ?
Quand est-ce qu’on mange ?
Tour de table …

Plan de cours

La formation:

- Introduction
- Getting started
- Images et Containers
- Ports
- Volumes
- Dockerfile
- Network

Pour aller plus loin:

- Docker Compose
- Docker Swarm
- Bonus...

Préparation de la formation

Docker - Evaluation Théorique

https://docs.google.com/forms/d/e/1FAIpQLSd1LCngH1ZKaVr8TK_oox-cBzDgnT2Vap_615Cm6pW5lX08oA/viewform?usp=sf_link

Créez un compte sur le Docker Hub

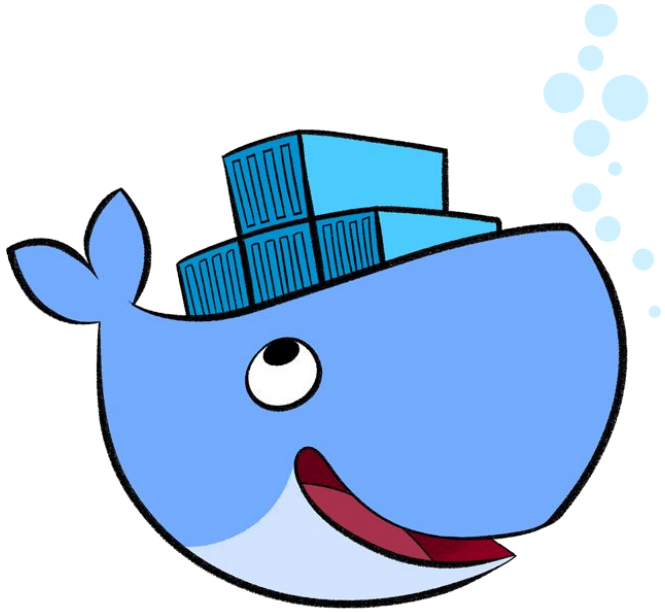
https://hub.docker.com/

```
$ git clone https://github.com/raab-d/Docker
```

```
$ docker run -ti --rm busybox echo "Hello world"
```

Introduction

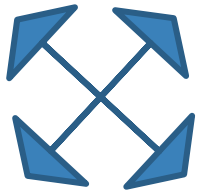
Qu’est-ce que Docker ?



La logistique



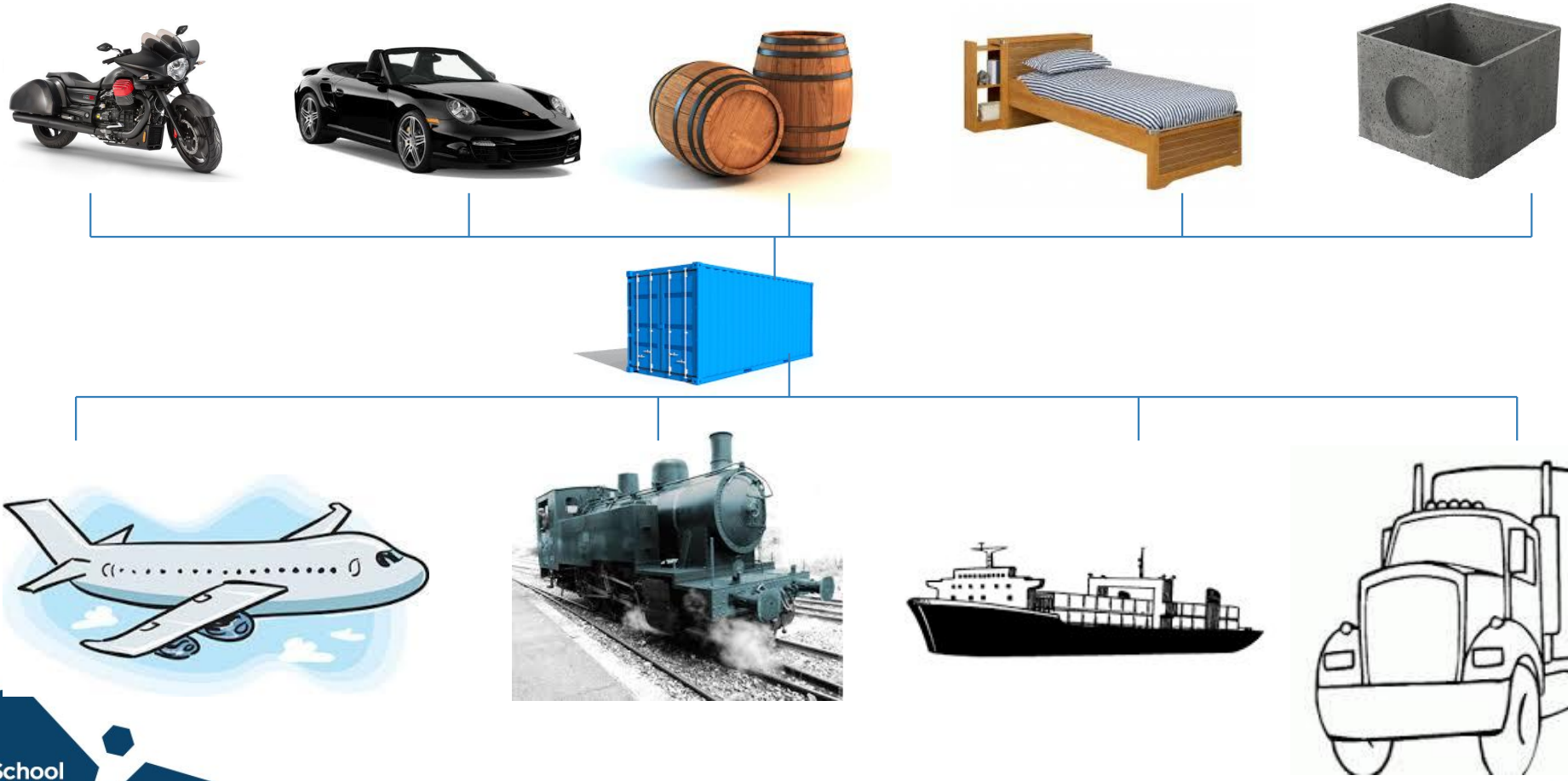




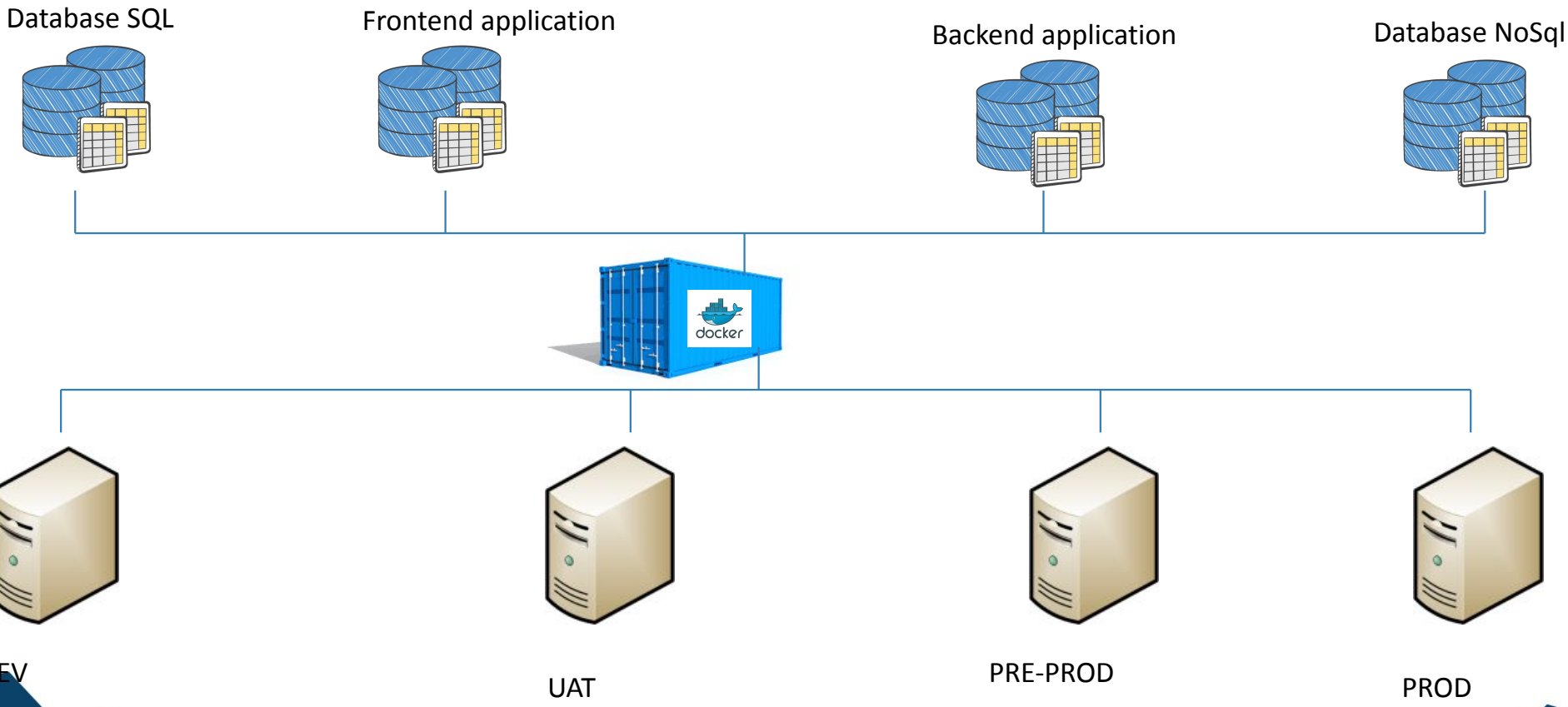
School

La logistique et les containers



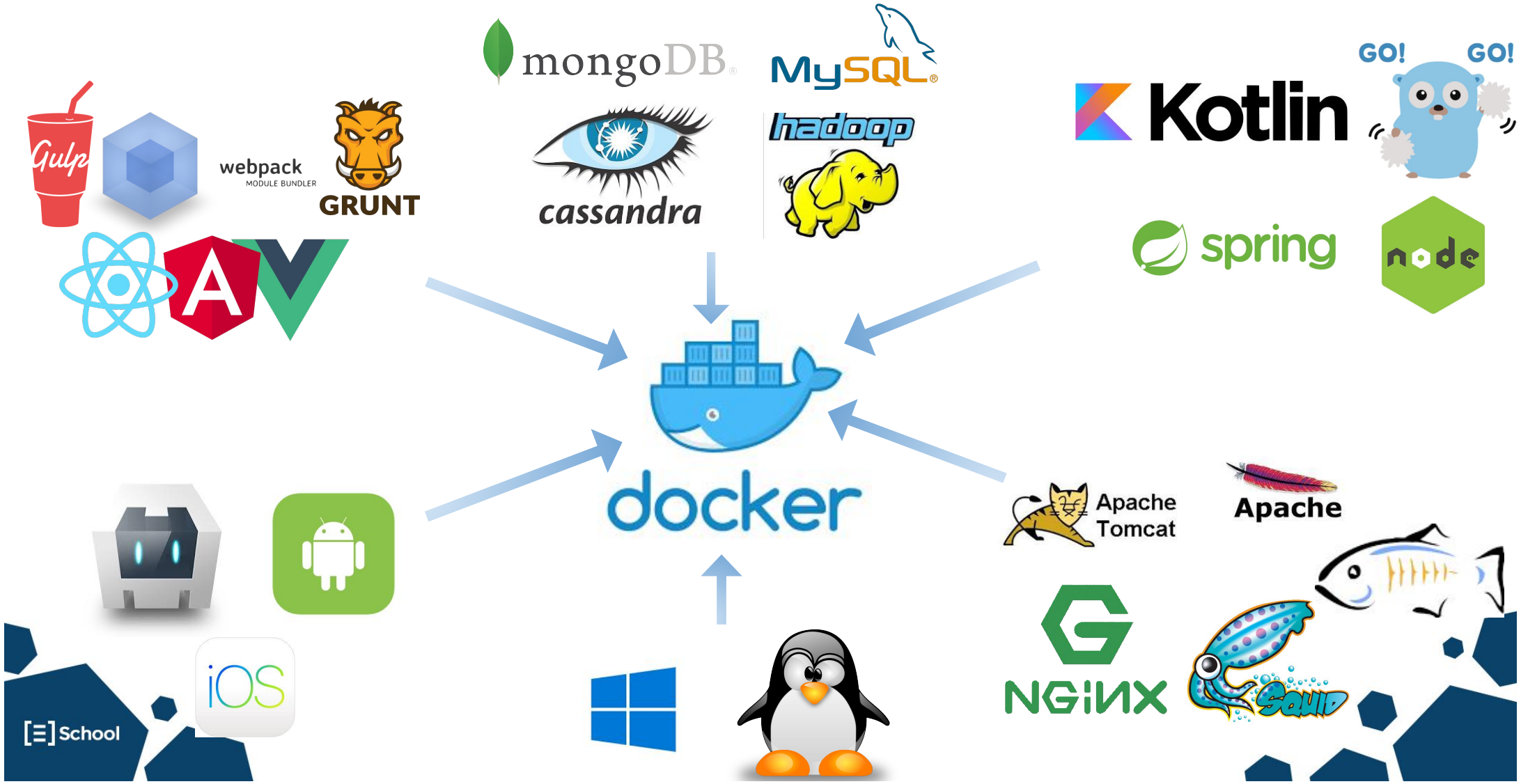
[E] School

L’informatique et les containers



[E] School

Qu’y mettons-nous ?



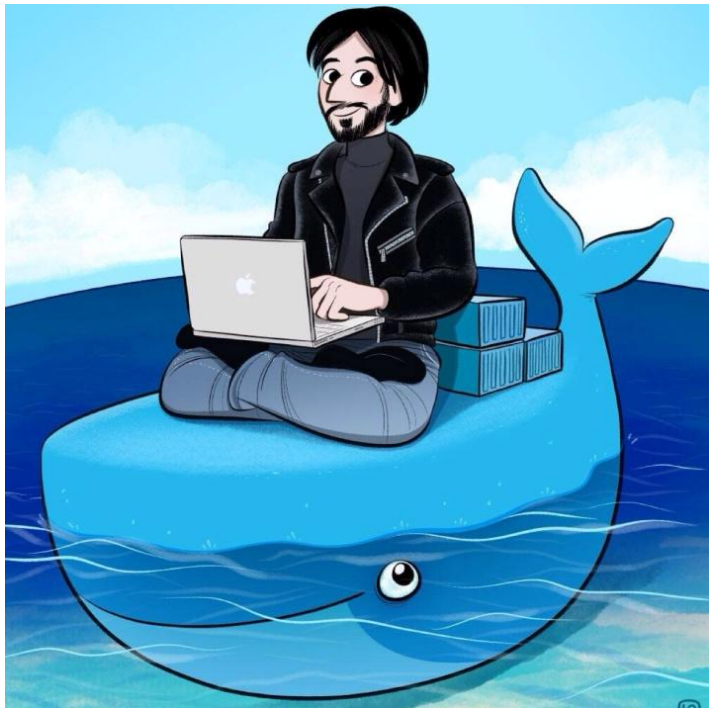
Qu’est-ce que Docker ?

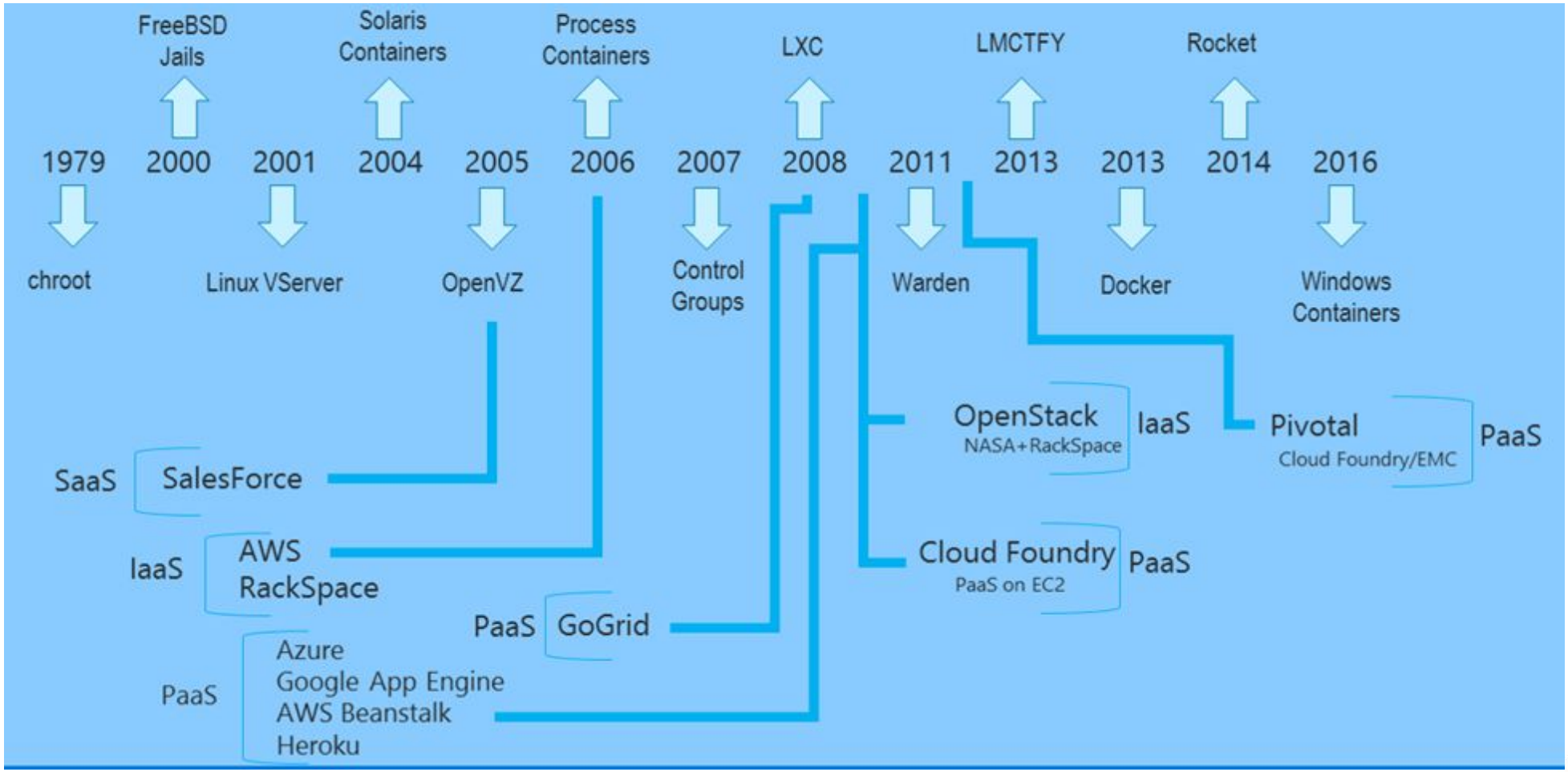
Docker permet de **packager une application**
avec **l’ensemble de ses dépendances**
dans une **unité standardisée** pour le déploiement de logiciels :

les **CONTAINERS**

D’où vient Docker ?

- Solomon Hykes @DotCloud
- Programmé en Go
- Open source
- 1ère version : 13 mars 2013




FreeBSD Jails
Solaris Containers
Process Containers
LXC
LMCTFY
Rocket
1979
2000
2001
2004
2005
2006
2007
2008
2011
2013
2013
2014
2016
chroot
Linux VServer
OpenVZ
Control Groups
Warden
Docker
Windows Containers
SaaS
SalesForce
IaaS
AWS
RackSpace
PaaS
GoGrid
PaaS
Azure
Google App Engine
AWS Beanstalk
Heroku
OpenStack
NASA+RackSpace
IaaS
Pivotal
Cloud Foundry/EMC
PaaS
Cloud Foundry
PaaS on EC2
PaaS

Introduction...

Getting started

Image vs Containers

Classes vs instances

```java
public class Point2D {

    private final int x;

    private final int y;

    public Point2D(int x, int y) {
        this.x = x;
        this.y = y;
    }

}
```

```java
public class Main {

    public static void main(String[] args) {

        Point2D point2D = new Point2D( x: 2,  y: 4);

    }
}
```

Les images

- Listez les images présentes sur votre machine :
  - **`> docker image ls`**

- Récupérez l'image "busybox" :
  - **`> docker image pull busybox`**

Les Containers

- Listez les containers actifs sur votre machine :
  - `> docker container ls`

- Listez tous les container sur votre machine :
  - `> docker container ls -a`

Premières exécutions

- Instanciez un container “Hello world” à partir de l’image busybox :
  - **`> docker container run`** busybox **`echo "Hello World"`**
- Instanciez un shell interactif à partir de l’image busybox :
  - **`> docker container run`** -i -t busybox
- Exécutez un “Hello world” dans ce shell puis quittez le container :
  - **`$ echo "Hello World"`**
  - **`$ exit`**

Containers éphémères

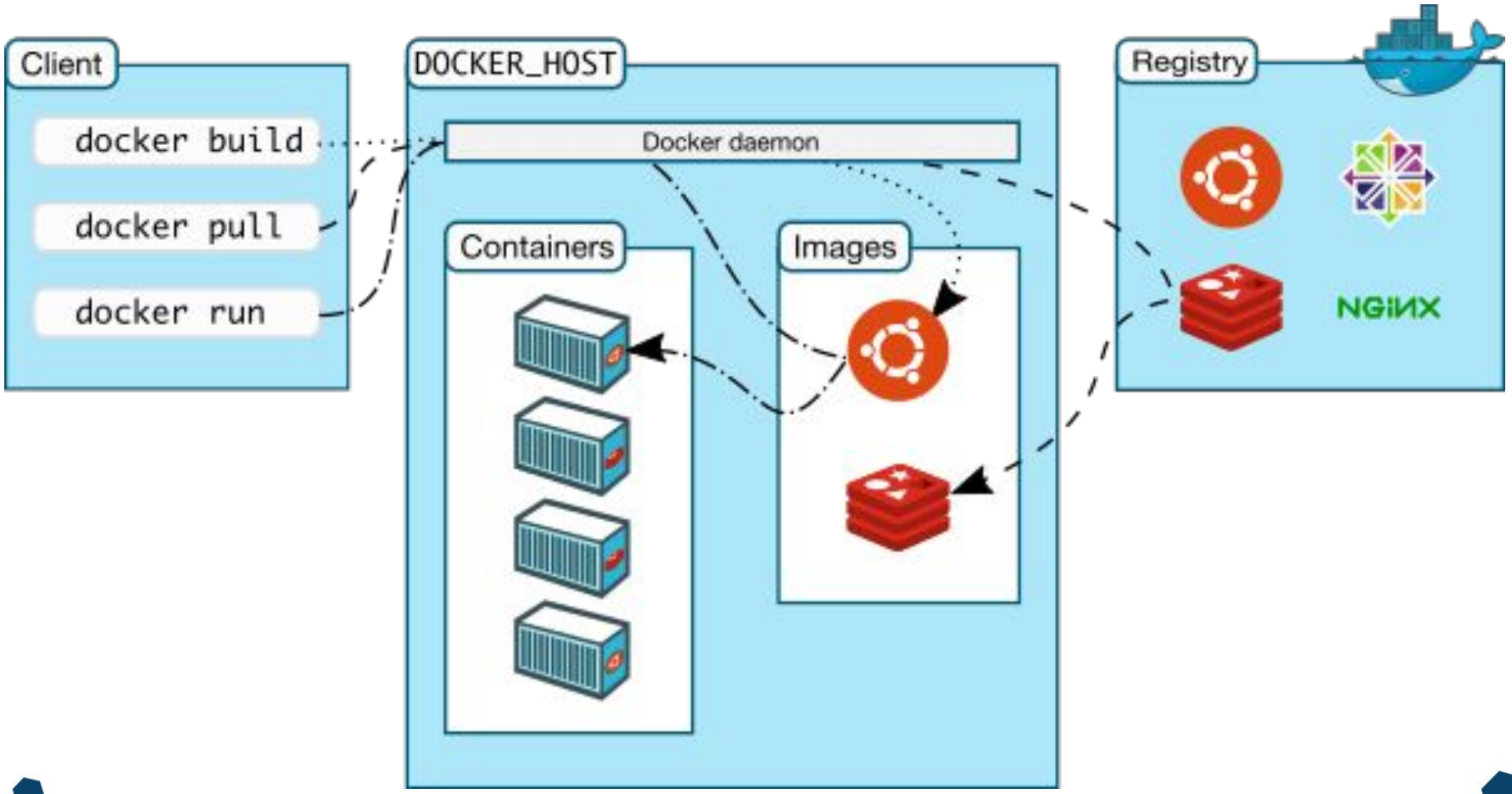
- Instanciez un image busybox interactif, supprimé à l'arrêt (**--rm**) :
  - `> docker container run --rm -it busybox`
- Dans ce container, créez des fichiers, puis quittez :
  - `$ echo "hello" > /docker-test.txt`
  - `$ exit`
- Recréez un container identique et affichez le contenu du fichier :
  - `> docker container run --rm -it busybox`
  - `$ cat /docker-test.txt`

Nettoyage des images

- Listez les images puis supprimez l'image busybox :
  - `> docker image ls`
  - `> docker image rm busybox`

- **Pour info**, pour supprimer toutes les images non utilisées par des containers :
  - `> docker image prune`

Conclusion


Client
docker build
docker pull
docker run
DOCKER_HOST
Docker daemon
Containers
Images
Registry
NGINX

Docker “management commands”

$ docker <OBJECT> <COMMAND>

image	ls, pull, rm, prune	Manage images
container	ls, run, stop, rm, prune	Manage containers

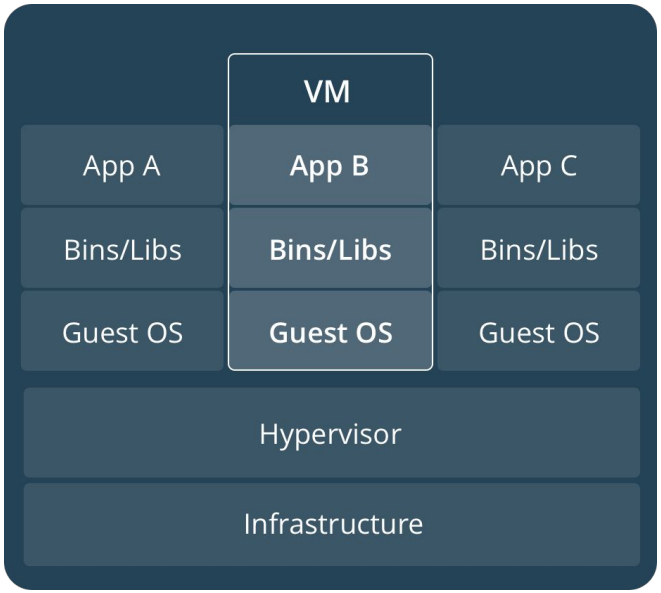
Lab 1 hands on docker

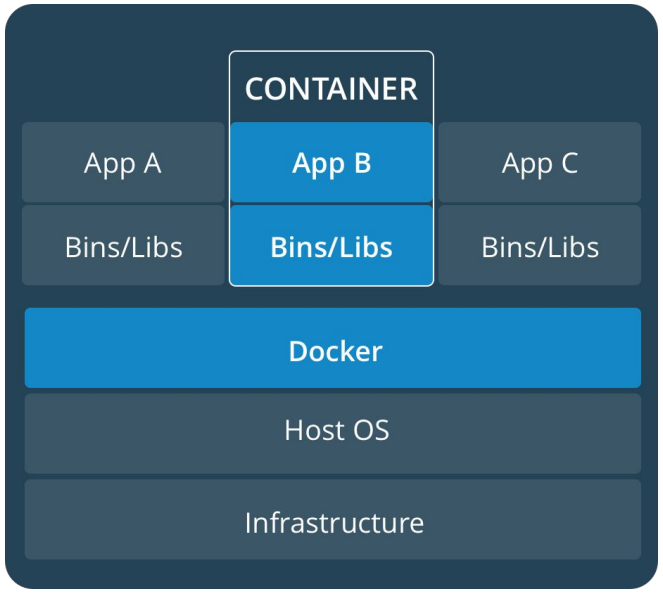
Introduction

Getting Started

Images et Containers

VM vs Containers





VM et Containers

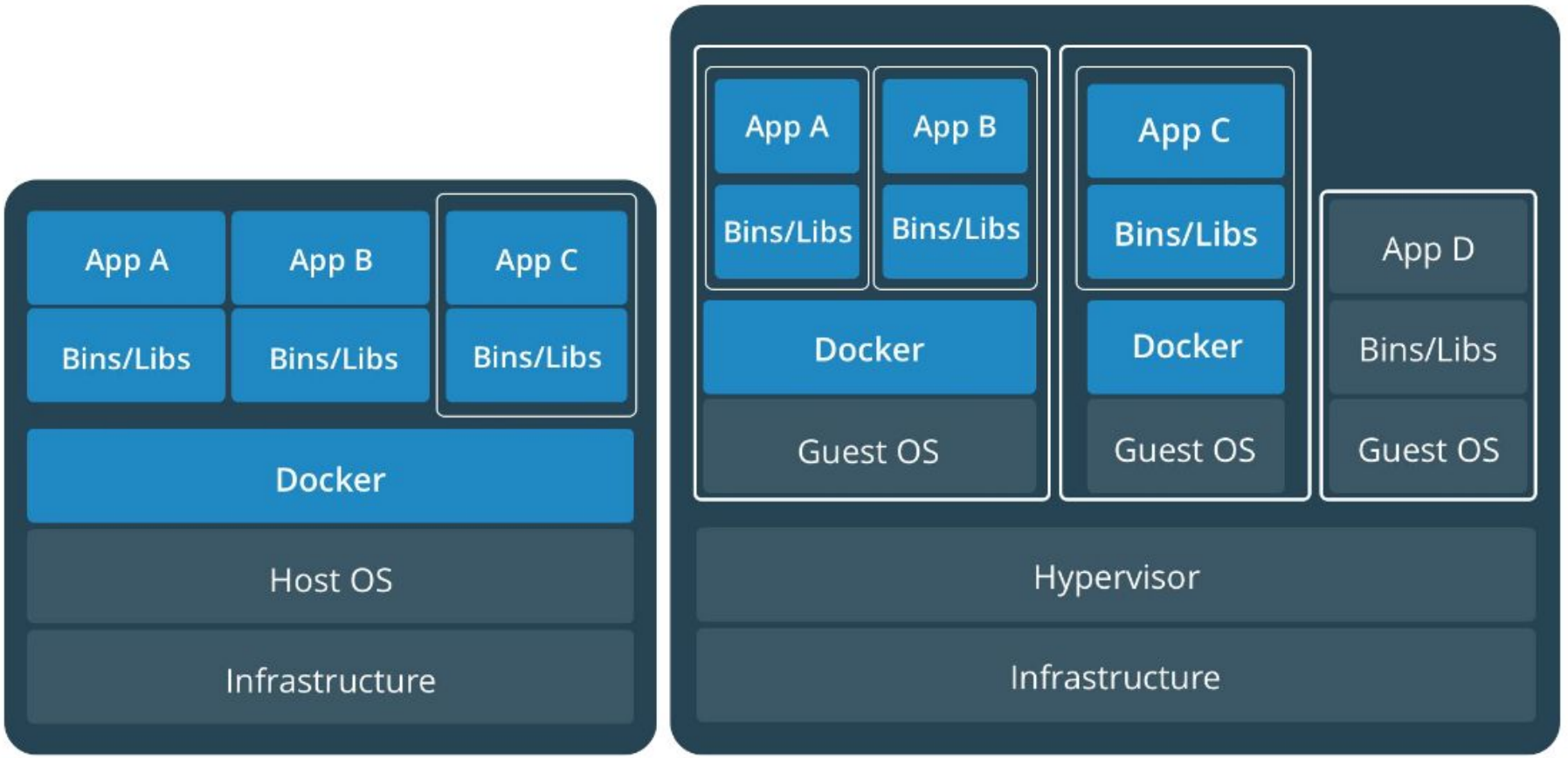
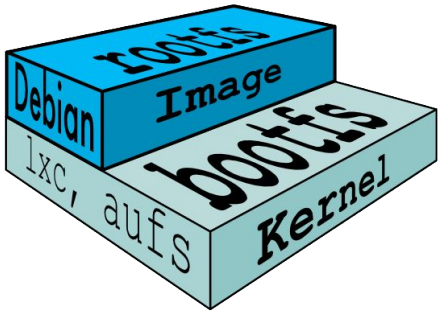
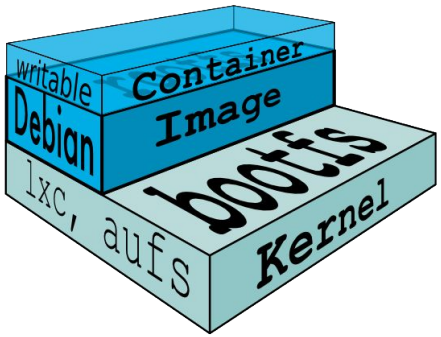


Image layers

- Image en lecture seule → immutabilité
- Copy-On-Write strategy





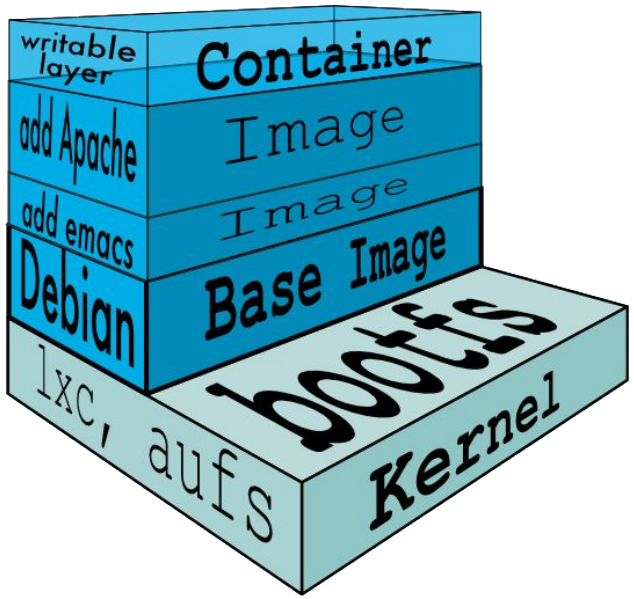
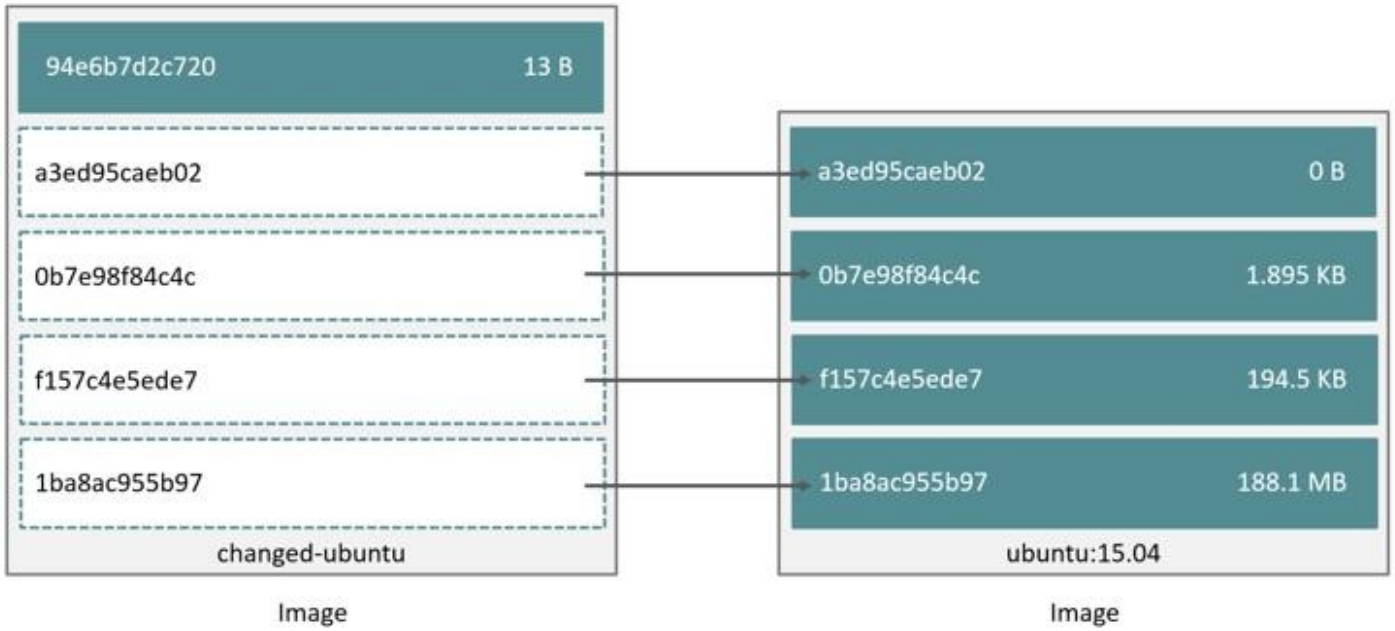


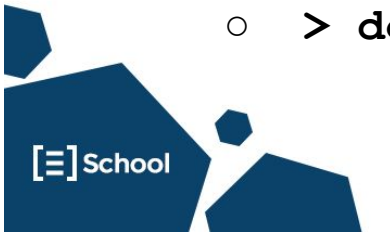
Image layers

- Espace disque
- Performances



Votre première image

- Récupérez une image alpine et lancez-là en mode interactif
  - `> docker image pull alpine:3.5`
  - `> docker container run -it --name node_ctn alpine:3.5`
- Installez nodejs puis quittez le container :
  - `$ apk add --update nodejs`
  - `$ exit`
- Créez une nouvelle image à partir du container :
  - `> docker container commit node_ctn mynodejs`



Partagez des images

- Taggez votre image :
  - **> docker image tag** mynodejs <dockerHubId>**/nodejs:1.0**
- Listez vos images locales. Que constatez-vous ?
  - **> docker image ls**
- Publiez votre image vers votre dépôt sur le Hub Docker
  - **> docker login**
  - **> docker image push** <dockerHubId>**/nodejs:1.0**

Images et tags

- Image ID = sha256 du contenu
- Tag :
  - pour identifier facilement une image
  - pour déterminer le dépôt d'origine

`[[`registry url`/]`username`/]`image`[:`tag`|@sha256]`

`hub.docker.com/library/`node`:latest`

`hub.docker.com/`zbbfufu/node:8.0

registry.sfeir.com:9000/taiebm/cordova:5.0-test

Docker “management commands”

$ docker <OBJECT> <COMMAND>

image	ls, pull, rm, prune, **push, tag**	Manage images
container	ls, run, stop, rm, prune, **commit**	Manage containers

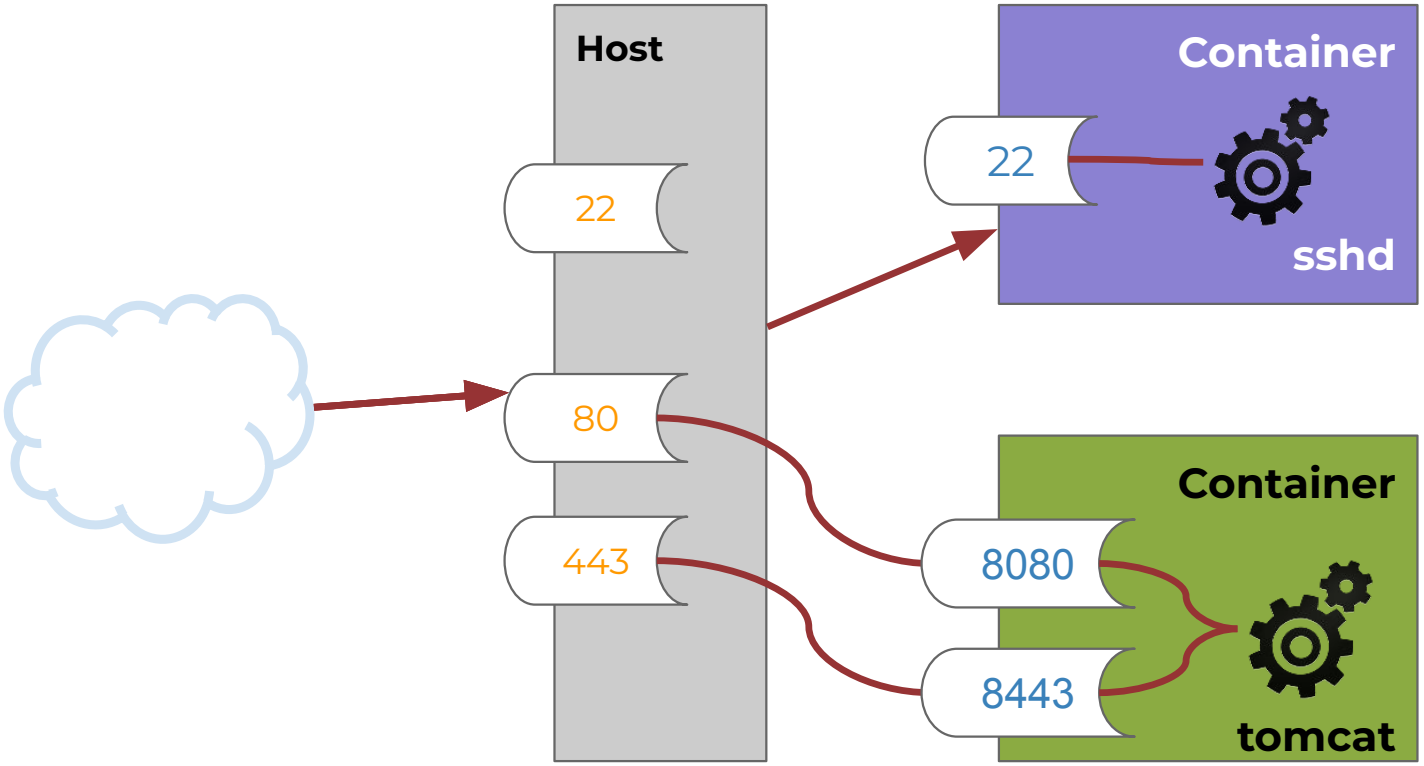
Introduction

Getting Started

Images et Containers

Ports

Port binding

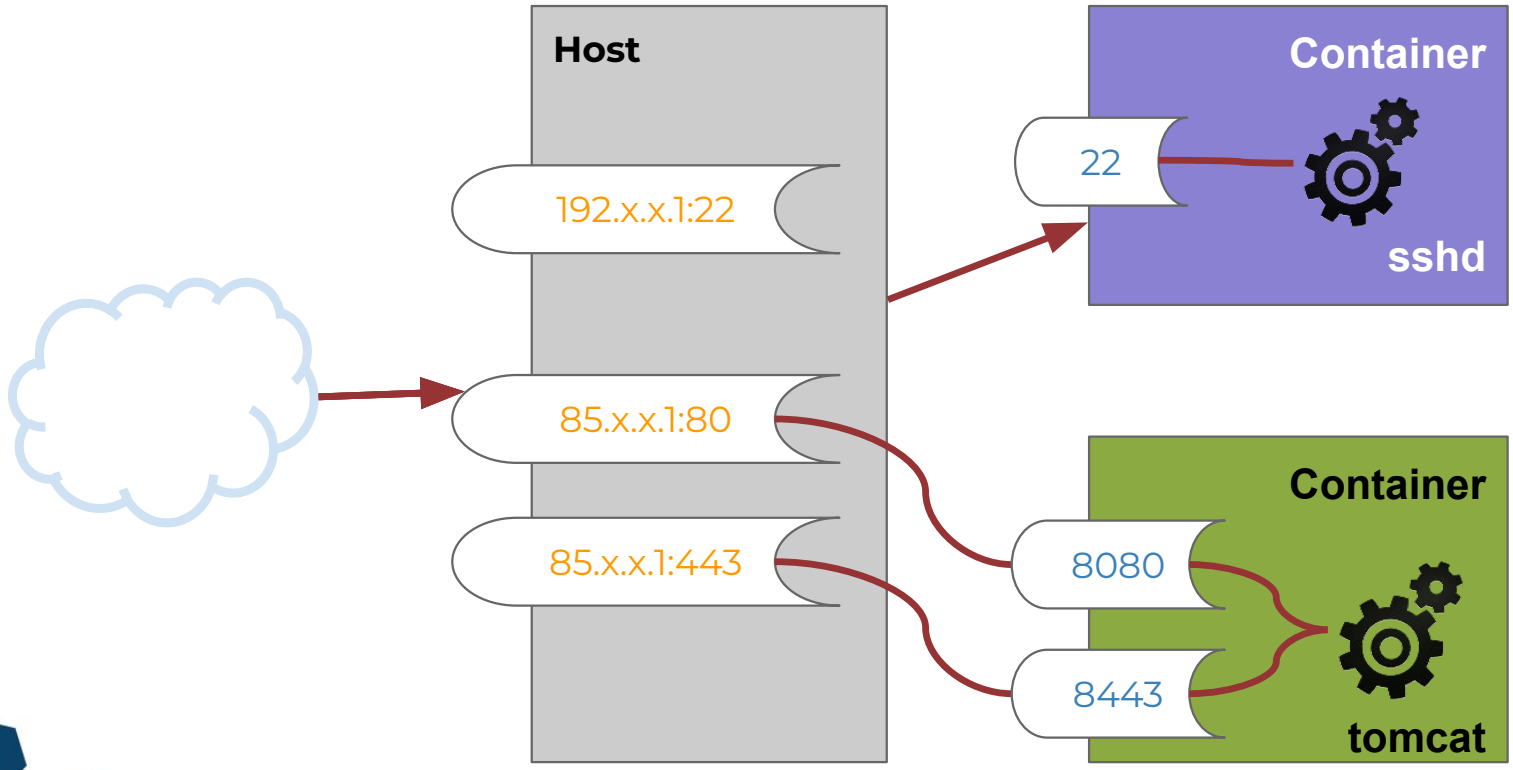


-p 22:22

-p 80:8080

-p 443:8443

Port binding



-p 192.x.x.1:22:22

-p 85.x.x.1:80:8080

-p 85.x.x.1:443:8443

[E] School

Publication de ports

- Récupérez l'image couchdb taguée 2.1 :
  - **`> docker image pull couchdb:2.1`**

- Instanciez un container couchdb1 détaché en exposant le port 5984 sur le port 5984 du host :
  - **`> docker container run --name couchdb1 -d`** -p 5984:5984 **`couchdb:2.1`**

- Ouvrez votre navigateur sur l'url http://localhost:5984 , CouchDB affiche sa version.

Docker “management commands”

$ docker <OBJECT> <COMMAND>

image	ls, pull, rm, prune, push, tag
container	ls, run **-p**, stop, rm, prune, commit

Lab 2 first docker image